1. What is the primary multidimensional container class provided by the NumPy package?

(A) list
(B) matrix
(C) ndarray
(D) vector

2. Which function creates a NumPy array initialized entirely with zeros?

(A) np.empty()
(B) np.zeros()
(C) np.null()
(D) np.zeroes()

3. What does the expression `np.arange(0, 10, 2)` generate?

(A) [0, 2, 4, 6, 8, 10]
(B) [0, 2, 4, 6, 8]
(C) [2, 4, 6, 8, 10]
(D) [0, 1, 2, 3, 4]

4. How is an evenly spaced array of 5 points between 0 and 1 (inclusive) generated?

(A) np.linspace(0, 1, 5)
(B) np.range(0, 1, 5)
(C) np.arange(0, 1, 0.2)
(D) np.interval(0, 1, 5)

5. Which attribute of an ndarray returns the total number of dimensions (axes)?

(A) shape
(B) size
(C) ndim
(D) itemsize

6. Which attribute reveals a tuple of integers indicating the size along each dimension?

(A) dim
(B) shape
(C) size
(D) len

7. What does `arr.size` return for an array with shape (3, 4)?

(A) 7
(B) 12
(C) 3
(D) 4

8. Which attribute returns the total memory consumed by a single array element in bytes?

(A) itemsize
(B) nbytes
(C) bytes
(D) bitsize

9. What is the result of `np.eye(3)`?

(A) 3x3 array of ones
(B) 3x3 identity matrix with ones on main diagonal
(C) 3x3 array of random numbers
(D) 1D array of length 3

10. What is the default floating-point data type in NumPy on 64-bit systems?

(A) float16
(B) float32
(C) float64
(D) float128

11. Which function creates an identity matrix with ones on diagonal and zeros elsewhere?

(A) np.diag()
(B) np.identity()
(C) np.ones()
(D) np.zeros()

12. What does `np.full((2, 2), 7)` return?

(A) 2x2 array filled with 7
(B) 1D array [7, 7]
(C) Matrix with 7 on diagonal
(D) Error

13. How do you cast an existing array `arr` to 32-bit floating point?

(A) arr.to_float()
(B) arr.astype(np.float32)
(C) float(arr)
(D) np.cast(arr, 32)

14. What is the result of basic arithmetic `a + b` between two NumPy arrays of shape (4,)?

(A) Matrix product
(B) Element-wise addition
(C) Concatenation into length 8
(D) Dot product

15. What is the result of `np.array([1, 2, 3]) * 2`?

(A) [1, 2, 3, 1, 2, 3]
(B) [2, 4, 6]
(C) TypeError
(D) 6

16. What does `arr[1, 2]` retrieve from a 2D ndarray?

(A) Element at row index 1, column index 2
(B) Subarray of 2 rows
(C) Element at row 2, col 1
(D) Slice from 1 to 2

17. What does `arr[:, 0]` extract from a 2D matrix?

(A) First row
(B) First column
(C) Main diagonal
(D) Entire matrix

18. What does `arr[0, :]` extract from a 2D matrix?

(A) First row
(B) First column
(C) Last column
(D) Single scalar

19. Which function changes array shape without modifying its data?

(A) np.change_shape()
(B) np.reshape()
(C) np.reorder()
(D) np.resize()

20. What does `arr.reshape(-1, 1)` do to a 1D array of 6 elements?

(A) Produces a 2D column vector of shape (6, 1)
(B) Flattens the array
(C) Produces shape (1, 6)
(D) Throws error

21. Which method flattens a multi-dimensional array and returns a contiguous 1D copy?

(A) ravel()
(B) flatten()
(C) squeeze()
(D) flat()

22. What is the difference between `ravel()` and `flatten()`?

(A) flatten() returns a copy; ravel() returns a view whenever possible
(B) ravel() returns a copy; flatten() returns a view
(C) ravel() only works on 3D
(D) No difference

23. What does `np.sum(arr, axis=0)` compute for a 2D matrix?

(A) Sum across entire matrix
(B) Sum of each column (down rows)
(C) Sum of each row (across columns)
(D) Cumulative sum

24. What does `np.sum(arr, axis=1)` compute for a 2D matrix?

(A) Sum of each row (across columns)
(B) Sum of each column
(C) Total sum
(D) Average

25. Which function finds the index of the maximum value in an ndarray?

(A) np.max()
(B) np.argmax()
(C) np.findmax()
(D) np.maximum()

26. What does `np.min(arr)` calculate?

(A) Smallest value in the entire array
(B) Indices of minimums
(C) Zero elements
(D) Absolute difference

27. Which function computes the arithmetic mean along a specified axis?

(A) np.average()
(B) np.mean()
(C) np.median()
(D) np.mode()

28. What does `np.std(arr)` compute?

(A) Standard deviation
(B) Variance
(C) Standard error
(D) Skewness

29. What does `np.var(arr)` compute?

- **(A)** Variance
- **(B)** Variable name
- **(C)** Standard deviation
- **(D)** Covariance

30. What does `np.transpose(arr)` or `arr.T` do to a 2D array?

- **(A)** Inverts the matrix
- **(B)** Permutes rows and columns (transposes)
- **(C)** Calculates determinant
- **(D)** Flattens array

31. What does the operator `@` perform between two 2D NumPy arrays?

- **(A)** Element-wise multiplication
- **(B)** Matrix multiplication (dot product)
- **(C)** Kronecker product
- **(D)** Cross product

32. What does `a * b` perform between two 2D arrays of identical shape?

- **(A)** Hadamard (element-wise) multiplication
- **(B)** Matrix multiplication
- **(C)** Outer product
- **(D)** Convolution

33. Which function generates an array of standard normally distributed random numbers (mean 0, std 1)?

- **(A)** np.random.random()
- **(B)** np.random.randn()
- **(C)** np.random.uniform()
- **(D)** np.random.randint()

34. How is reproducibility in random number generation established in modern NumPy?

- **(A)** np.random.seed(42) or np.random.default_rng(42)
- **(B)** np.random.freeze()
- **(C)** np.random.constant()
- **(D)** np.random.set()

35. What is the output of `np.clip(arr, 0, 5)`?

- **(A)** Discards elements outside [0, 5]
- **(B)** Limits array values such that values < 0 become 0 and values > 5 become 5
- **(C)** Resizes array to 5
- **(D)** Splits at 5

36. Which function checks if any element in a boolean array is True?

- **(A)** np.all()
- **(B)** np.any()
- **(C)** np.is_true()
- **(D)** np.contains()

37. Which function checks if every element in a boolean array is True?

- **(A)** np.any()
- **(B)** np.all()
- **(C)** np.true()
- **(D)** np.check()

38. What does `np.diff([1, 4, 9, 16])` produce?

- **(A)** [3, 5, 7]
- **(B)** [1, 3, 5, 7]
- **(C)** [4, 9, 16]
- **(D)** [3, 3, 3]

39. Which function calculates the cumulative sum of array elements?

- **(A)** np.sum()
- **(B)** np.cumsum()
- **(C)** np.cumprod()
- **(D)** np.trapz()

40. What does `np.squeeze(arr)` do?

- **(A)** Removes single-dimensional entries (dimensions of size 1) from the shape
- **(B)** Compresses array using zip
- **(C)** Shrinks memory size
- **(D)** Scales to range [0, 1]

41. What are the broadcasting rules when operating on two arrays of shapes (3, 1) and (1, 4)?

- **(A)** Incompatible shapes, raises ValueError
- **(B)** Broadcasts to shape (3, 4)
- **(C)** Broadcasts to shape (4, 3)
- **(D)** Produces 1D array of 12 elements

42. What happens when operating on arrays with shapes (5, 3) and (5,)?

- **(A)** Operates cleanly along axis 0
- **(B)** Incompatible trailing dimension: 3 vs 5, raises ValueError
- **(C)** Expands to (5, 5, 3)
- **(D)** Truncates to (5, 1)

43. Which function stacks arrays vertically along the first axis (row-wise)?

(A) np.hstack()
(B) np.vstack()
(C) np.dstack()
(D) np.concatenate(axis=1)

44. Which function stacks 1D arrays as columns into a 2D matrix?

(A) np.column_stack()
(B) np.row_stack()
(C) np.stack(axis=0)
(D) np.append()

45. What is the return type of `arr[arr > 0]` where `arr` is a 2D ndarray?

(A) A 2D masked array
(B) A 1D array containing only positive elements
(C) A boolean matrix
(D) A tuple of coordinates

46. What does `np.where(condition, x, y)` do?

(A) Returns array of elements selected from x or y depending on condition
(B) Finds index where condition is True
(C) Filters elements matching condition
(D) Replaces NaNs

47. When `np.where(condition)` is called with only ONE argument, what does it return?

(A) A boolean array
(B) A tuple of coordinate index arrays where condition is True
(C) Count of True elements
(D) Filtered values

48. What does `np.isin(element, test_elements)` do?

(A) Returns boolean array of whether each element is in test_elements
(B) Sorts elements
(C) Computes intersection
(D) Checks identity

49. Which function splits an array into multiple sub-arrays horizontally (column-wise)?

(A) np.vsplit()
(B) np.hsplit()
(C) np.dsplit()
(D) np.split(axis=0)

50. What is a universal function (`ufunc`) in NumPy?

(A) A function that operates on ndarrays in an element-by-element fashion compiled in C
(B) A GUI widget
(C) A machine learning pipeline
(D) A string formatting tool

51. Which method of a ufunc computes the cumulative application of the operator along an axis?

(A) reduce()
(B) accumulate()
(C) reduceat()
(D) outer()

52. What does `np.add.reduce([1, 2, 3, 4])` compute?

(A) 10
(B) [1, 3, 6, 10]
(C) [1, 2, 3, 4]
(D) 0

53. What does `np.multiply.outer([1, 2], [10, 20])` produce?

(A) [[10, 20], [20, 40]]
(B) [10, 40]
(C) 50
(D) [10, 20, 20, 40]

54. What is the purpose of `np.vectorize(pyfunc)`?

(A) Compiles python code into GPU CUDA kernel
(B) Provides a convenient vectorized interface over a python scalar function without C speedups
(C) Generates assembly code
(D) Parallelizes over multi-threading

55. How does vectorization in native NumPy achieve 50x-100x speedups over pure Python loops?

(A) Bypasses CPython interpreter overhead and utilizes C loops and SIMD vector registers
(B) Uses thread locking
(C) Compiles to Javascript
(D) Eliminates all memory accesses

56. What does basic slicing `sub = arr[1:3]` return in NumPy?

(A) An independent memory copy
(B) A view of the original array sharing underlying buffer
(C) An unchangeable tuple
(D) A linked list

57. What happens to the original array if you execute `sub = arr[1:3]; sub[0] = 999`?

(A) Original `arr` is NOT modified
(B) Original `arr` is modified at index 1 because slicing returns a view
(C) Raises ReadOnlyError
(D) Allocates a new buffer

58. What type of indexing ALWAYS creates a copy instead of a view?

(A) Basic integer slicing `arr[1:5]`
(B) Advanced indexing (using integer array or boolean masks)
(C) Strided slicing `arr[::2]`
(D) Ellipsis slicing `arr[...]`

59. What is the purpose of the Ellipsis object `...` in NumPy indexing?

(A) Represents all dimensions not explicitly specified: e.g. `arr[..., 0]`
(B) Indicates infinity
(C) Creates a syntax error
(D) Truncates data

60. How do you add a new axis of size 1 to an array `arr` of shape (5,)?

(A) `arr[:, np.newaxis]` or `arr[:, None]`
(B) `arr.expand()`
(C) `arr.add_axis(1)`
(D) `np.insert(arr, 1)`

61. Which function computes numerical integration using the composite trapezoidal rule?

(A) np.integrate()
(B) np.trapz()
(C) np.quad()
(D) np.simps()

62. What does `np.gradient(f, dx)` calculate for a 1D discrete signal array?

(A) Numerical derivative using central differences in interior points
(B) Integral of f
(C) Fast Fourier Transform
(D) Root mean square

63. Which function computes the cross-correlation or convolution of two 1D sequences?

(A) np.convolve()
(B) np.correlate()
(C) Both np.convolve and np.correlate
(D) np.xcorr()

64. What mode parameter in `np.convolve(a, v, mode='same')` ensures output length matches the first signal?

(A) 'full'
(B) 'valid'
(C) 'same'
(D) 'circular'

65. What is the length of `np.convolve(x, h, mode='full')` where len(x)=N and len(h)=M?

(A) N + M
(B) N + M - 1
(C) max(N, M)
(D) min(N, M)

66. Which function finds unique elements of an array in sorted order?

(A) np.set()
(B) np.unique()
(C) np.distinct()
(D) np.filter_unique()

67. What additional information can `np.unique(arr, return_counts=True)` provide?

(A) Histogram frequency counts of each unique element
(B) Indices of first occurrences
(C) Inverse indices to reconstruct original array
(D) All of the above

68. Which function computes the Pearson product-moment correlation coefficients matrix?

(A) np.cov()
(B) np.corrcoef()
(C) np.correlate()
(D) np.pearson()

69. What does `np.pad(arr, (1, 1), mode='constant')` do to an array?

(A) Adds 1 row/col of zero padding around the borders
(B) Scales the array
(C) Trims edges by 1
(D) Multiplies by 1

70. How do you check if any elements in an array are NaN (Not a Number)?

(A) `arr == np.nan`
(B) `np.isnan(arr)`
(C) `arr.has_nan()`
(D) `np.is_null(arr)`

71. Why does `arr == np.nan` fail to detect NaN values in NumPy?

(A) Because according to IEEE 754 floating-point standards, NaN never compares equal to anything, including itself
(B) NumPy does not support NaN
(C) Requires strict typing
(D) Compiler optimization issue

72. Which function replaces NaN values with zero or another specified scalar in an array?

(A) np.replace_nan()
(B) np.nan_to_num()
(C) np.fill_nan()
(D) np.clean()

73. What does `np.tile(A, (2, 3))` do?

(A) Constructs an array by repeating A 2 times along rows and 3 times along columns
(B) Splits A into 6 tiles
(C) Reshapes A to 2x3
(D) Transposes A

74. What does `np.repeat(A, 3)` do compared to `np.tile(A, 3)`?

(A) Repeats each individual element 3 times sequentially
(B) Repeats the entire block A 3 times
(C) Multiplies by 3
(D) Reshapes array

75. Which function computes the root-mean-square (RMS) value of a discrete waveform `x`?

(A) `np.sqrt(np.mean(x**2))`
(B) `np.mean(np.sqrt(x))`
(C) `np.std(x)`
(D) `np.rms(x)`

76. What does `arr.strides` represent in a NumPy ndarray?

(A) The number of bytes to step in each dimension when moving to the next element
(B) The total count of rows
(C) The memory bus width
(D) The alignment offset

77. For a 2D float64 array of shape (3, 4) in standard C-contiguous order, what are its strides?

(A) (32, 8)
(B) (8, 32)
(C) (24, 8)
(D) (8, 8)

78. What is the memory difference between C-contiguous and Fortran-contiguous array ordering?

(A) C-order stores rows contiguously (last index changes fastest); Fortran-order stores columns contiguously (first index changes fastest)
(B) C-order is 32-bit; Fortran is 64-bit
(C) Fortran-order is immutable
(D) No difference in memory

79. Which function from `numpy.lib.stride_tricks` creates sliding window views without memory duplication?

(A) sliding_window_view() or as_strided()
(B) strided_copy()
(C) window_buffer()
(D) rolling_view()

80. Why is modifying the output of `as_strided()` extremely dangerous?

(A) It can write to arbitrary memory locations and trigger segmentation faults if strides exceed allocated boundaries
(B) It causes python syntax errors
(C) It changes endianness
(D) It freezes the CPU

81. Which submodule of NumPy handles matrix decompositions, inverses, and linear system solvers?

(A) np.linalg
(B) np.matrix
(C) np.solvers
(D) np.math

82. Which function computes the multiplicative inverse of a square non-singular matrix A?

(A) np.linalg.pinv(A)
(B) np.linalg.inv(A)
(C) np.inverse(A)
(D) A.invert()

83. What does `np.linalg.pinv(A)` compute for rectangular or singular matrices?

(A) Moore-Penrose pseudo-inverse using SVD
(B) Standard inverse
(C) Matrix cofactor
(D) Adjoint matrix

84. Which function solves the linear matrix equation $Ax = b$ efficiently without explicitly computing A^{-1}?

(A) np.linalg.inv(A) @ b
(B) np.linalg.solve(A, b)
(C) np.linalg.eval(A, b)
(D) np.linalg.det(A, b)

85. Why is `np.linalg.solve(A, b)` preferred over `np.linalg.inv(A) @ b` in numerical engineering?

- **(A)** It employs LU or Cholesky decomposition directly, requiring fewer FLOPs and offering substantially higher numerical stability
- **(B)** It uses GPU automatically
- **(C)** It returns integers
- **(D)** It supports strings

86. Which function computes the eigenvalues and right eigenvectors of a square matrix?

- **(A)** np.linalg.eig()
- **(B)** np.linalg.eigen()
- **(C)** np.linalg.values()
- **(D)** np.linalg.diag()

87. What is the return structure of `w, v = np.linalg.eig(A)`?

- **(A)** `w` is a 1D array of eigenvalues; `v` is a 2D array where columns are normalized eigenvectors
- **(B)** `w` is eigenvectors; `v` is eigenvalues
- **(C)** Two scalar values
- **(D)** A diagonal matrix

88. Which function performs Singular Value Decomposition (SVD) such that $A = U \Sigma V^H$?

- **(A)** np.linalg.svd()
- **(B)** np.linalg.decompose()
- **(C)** np.linalg.factor()
- **(D)** np.linalg.qr()

89. Which decomposition is computed by `np.linalg.qr(A)`?

- **(A)** Orthogonal matrix Q and Upper triangular matrix R
- **(B)** Lower and Upper triangular matrices
- **(C)** Eigenvalue decomposition
- **(D)** Spectral decomposition

90. Which function computes the 1D discrete Fourier transform of a real or complex sequence?

- **(A)** np.fft.fft()
- **(B)** np.fft.rfft()
- **(C)** np.fourier.transform()
- **(D)** np.signal.dft()

91. What does `np.fft.rfft()` compute compared to `np.fft.fft()`?

- **(A)** One-sided FFT for strictly real-valued inputs, exploiting Hermitian symmetry to return only $N/2 + 1$ frequency bins
- **(B)** Reverse FFT
- **(C)** Recursive FFT
- **(D)** Real part only

92. What does `np.fft.fftfreq(N, d=1/fs)` generate?

- **(A)** Discrete Fourier Transform sample frequencies corresponding to each FFT bin
- **(B)** Magnitudes of spectrum
- **(C)** Phase angles
- **(D)** Nyquist bandwidth

93. What does `np.fft.fftshift(X)` do to a two-sided frequency spectrum?

- **(A)** Shifts zero-frequency component to the center of the spectrum array
- **(B)** Removes DC offset
- **(C)** Inverts phase
- **(D)** Computes absolute values

94. What underlying low-level linear algebra libraries power NumPy's high-speed operations?

- **(A)** BLAS (Basic Linear Algebra Subprograms) and LAPACK
- **(B)** OpenGL and DirectX
- **(C)** POSIX pthreads only
- **(D)** Win32 GDI

95. What is a memory-mapped array in NumPy created with `np.memmap`?

- **(A)** An ndarray that maps large on-disk binary files directly into virtual memory space without loading the entire file into RAM
- **(B)** RAM drive storage
- **(C)** GPU memory buffer
- **(D)** Compressed archive

96. What is a structured array (record array) in NumPy?

- **(A)** An ndarray whose dtype is defined as a sequence of named fields of heterogeneous types, similar to a C struct
- **(B)** A binary search tree
- **(C)** A hash table
- **(D)** An XML document

97. What is the time complexity of the Cooley-Tukey Radix-2 FFT algorithm implemented in `np.fft.fft` for length N?

- **(A)** $O(N^2)$
- **(B)** O(N log N)
- **(C)** O(log N)
- **(D)** O(N)

98. What does `np.einsum('ij,jk->ik', A, B)` perform using Einstein summation convention?

- **(A)** Matrix multiplication of 2D matrices A and B
- **(B)** Element-wise multiplication
- **(C)** Trace of matrix
- **(D)** Outer product

99. What does `np.einsum('ii->', A)` calculate?

(A) Trace (sum of diagonal elements) of matrix A
(B) Determinant
(C) Norm
(D) Inverse

100. What is the result of `np.shares_memory(a, b)`?

(A) Returns True if two arrays share the same memory buffer (or overlap)
(B) Returns memory size
(C) Copies buffer
(D) Synchronizes cache

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the internal architecture of NumPy's `ndarray` compared to a standard Python list, focusing on memory layout and cache locality.

Q2. Formulate the mathematical rules governing NumPy array broadcasting and demonstrate with worked 2D/3D examples.

Q3. Explain array memory strides, C-order vs. Fortran-order, and how `strides` enable zero-copy transposition and slicing.

Q4. Analyze the performance and memory implications of slicing views versus advanced indexing copies in NumPy.

Q5. Demonstrate the mathematical formulation and NumPy implementation of 1D signal convolution and FIR filtering.

Q6. Explain the Discrete Fourier Transform (DFT) mathematical formulation and spectral analysis using `np.fft`.

Q7. Describe matrix decompositions in `np.linalg`: LU, QR, Cholesky, and Singular Value Decomposition (SVD).

Q8. Demonstrate solving a system of linear equations $Ax = b$ using `np.linalg.solve` and discuss numerical condition number.

Q9. Explain eigenvalues and eigenvectors in circuit stability and state-space analysis using `np.linalg.eig`.

Q10. Detail universal function (`ufunc`) mechanics in NumPy: element-wise execution, type casting, and inner loops.

Q11. Demonstrate structured arrays and record arrays in NumPy for heterogeneous sensor data acquisition.

Q12. Explain `np.memmap` for memory-mapped files and how to stream gigabyte-scale radar datasets.

Q13. Demonstrate the use of `np.einsum` (Einstein Summation) for multidimensional tensor operations.

Q14. Formulate numerical differentiation techniques in NumPy: Forward, Backward, and Central differences with error orders.

Q15. Explain the composite Trapezoidal and Simpson's rules for numerical integration using NumPy.

Q16. Describe random variable sampling in modern NumPy using `Generator` and `PCG64` bit generators.

Q17. Demonstrate fast search and insertion in sorted NumPy arrays using `np.searchsorted`.

Q18. Explain array concatenation, splitting, and stacking along custom axes with mathematical diagrams.

Q19. Demonstrate numerical root-finding using companion matrices and polynomial roots with `np.roots`.

Q20. Explain masked arrays in NumPy (`numpy.ma`) for handling corrupted ADC sensor readings and missing data.

Q21. Implement an end-to-end discrete Kalman Filter in NumPy for tracking noisy radar trajectory measurements.

Q22. Write a vectorized 2D Finite-Difference Time-Domain (FDTD) wave equation solver using NumPy array operations.

Q23. Derive the Cooley-Tukey Radix-2 Decimation-In-Time (DIT) FFT algorithm and analyze its computational savings.

Q24. Implement a sliding-window Short-Time Fourier Transform (STFT) in NumPy using `as_strided` and window functions.

Q25. Demonstrate low-rank matrix approximation via Truncated Singular Value Decomposition (TSVD) for signal denoising.

Q26. Implement a vectorized Digital Phase-Locked Loop (DPLL) phase error detector and loop filter in NumPy.

Q27. Explain the implementation and performance optimization of vectorized N-body gravitational or electromagnetic simulation.

Q28. Detail NumPy memory management: PyArrayObject, buffer sharing, views, base attributes, and garbage collection.

Q29. Explain the implementation of custom universal functions using Numba `@vectorize` with hardware SIMD acceleration.

Q30. Implement an adaptive Least Mean Squares (LMS) adaptive filter in NumPy for active noise cancellation.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1: [C]
Q2: [B]
Q3: [B]
Q4: [A]
Q5: [C]
Q6: [B]
Q7: [B]
Q8: [A]
Q9: [B]
Q10: [C]
Q11: [B]
Q12: [A]
Q13: [B]
Q14: [B]
Q15: [B]
Q16: [A]
Q17: [B]
Q18: [A]
Q19: [B]
Q20: [A]

Q21: [B]
Q22: [A]
Q23: [B]
Q24: [A]
Q25: [B]
Q26: [A]
Q27: [B]
Q28: [A]
Q29: [A]
Q30: [B]
Q31: [B]
Q32: [A]
Q33: [B]
Q34: [A]
Q35: [B]
Q36: [B]
Q37: [B]
Q38: [A]
Q39: [B]
Q40: [A]

Q41: [B]
Q42: [B]
Q43: [B]
Q44: [A]
Q45: [B]
Q46: [A]
Q47: [B]
Q48: [A]
Q49: [B]
Q50: [A]
Q51: [B]
Q52: [A]
Q53: [A]
Q54: [B]
Q55: [A]
Q56: [B]
Q57: [B]
Q58: [B]
Q59: [A]
Q60: [A]

Q61: [B]
Q62: [A]
Q63: [C]
Q64: [C]
Q65: [B]
Q66: [B]
Q67: [D]
Q68: [B]
Q69: [A]
Q70: [B]
Q71: [A]
Q72: [B]
Q73: [A]
Q74: [A]
Q75: [A]
Q76: [A]
Q77: [A]
Q78: [A]
Q79: [A]
Q80: [A]

Q81: [A]
Q82: [B]
Q83: [A]
Q84: [B]
Q85: [A]
Q86: [A]
Q87: [A]
Q88: [A]
Q89: [A]
Q90: [A]
Q91: [A]
Q92: [A]
Q93: [A]
Q94: [A]
Q95: [A]
Q96: [A]
Q97: [B]
Q98: [A]
Q99: [A]
Q100: [A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION D: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS [illegible] and cache localit

- Axis 0: 4 vs 1 -> One is 1 (Compatible, B broadcasts from 1 to 4)
- Resulting Shape: (4, 3, 6).

[illegible]

>>Solution to Question 6: Explain the mathematical formulation of 1D signal convolution and filtering.

Problem Statement: Demonstrate the Discrete Fourier Transform (DFT) and Numerical implementation using FFT.

```
# Apply Hanning window to suppress spectral leakage
windowed = sig * np.hanning(N)
X = np.fft.rfft(windowed)
freqs = np.fft.rfftfreq(N, d=1/fs)

# Single-sided magnitude spectrum in dBV
magnitude = 20 * np.log10(np.abs(X) / (N / 2))
```

The use of `rfft()` exploits Hermitian symmetry $X[k] = X^*[N-k]$ for real-valued signals, computing only $N/2 + 1$ bins, halving memory and computational FLOPs.

>> [illegible] condition number.

```
])
b = np.array([24.0, 0.0, -12.0])

# Solve system via LU decomposition directly
x = np.linalg.solve(A, b)
```

Numerical Condition Number $\kappa(A)$:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\| = \frac{\sigma_{\max}}{\sigma_{\min}}$$

Calculated via `np.linalg.cond(A)`.

- If $\kappa(A) \approx 1$, the matrix is well-conditioned; errors in $b$ do not amplify.
- If $\kappa(A) \gg 10^8$, the matrix is ill-conditioned (near singular), and numerical round-off errors can completely destroy the solution accuracy.

[illegible]

array of function pointers for an exact match. If an exact match is missing, NumPy casts inputs according to safe casting hierarchy rules (e.g. `int32 -> float64`).
3. Inner Loops: Once types are resolved, the ufunc executes a tightly optimized C loop iterating through pointers using element strides, completely bypassing Python C-API interpreter overhead.
4. Ufunc Methods: Every ufunc supports built-in reduction operations: `reduce()`, `accumulate()`, `reduceat()`, and `outer()`.

[illegible]

simultaneously, sharing OS page cache pages without duplicating physical RAM.

```python
# Map a 20 GB raw I/Q radar sample binary file
radar_data = np.memmap('raw_radar.dat', dtype=np.complex64, mode='r', shape=(100000, 2048))
# Process frame 500 without loading the remaining 19.9 GB
frame_spectrum = np.fft.fft(radar_data[500, :])
```

[illegible] with error orders.

I

[illegible]

[Detailed Step-by-Step Answer]:
`np.searchsorted(a, v, side='left')` uses binary search to locate indices where elements $v$ should be inserted into sorted array $a$ to maintain sorted order:

```python
import numpy as np
thresholds = np.array([0.5, 1.0, 2.0, 3.3, 5.0])
voltages = np.array([0.1, 0.7, 2.5, 4.0, 6.0])

# Fast binary quantization
bin_indices = np.searchsorted(thresholds, voltages)
# Returns: [0, 1, 3, 4, 5]
```

Time Complexity: $\mathcal{O}(M \log N)$ where $M$ is the number of query points and $N$ is array length, orders of magnitude faster than linear masking for quantization and lookup tables.

[illegible]

$$C = \begin{bmatrix} -c_1/c_0 & -c_2/c_0 & \dots & -c_{n-1}/c_0 & -c_n/c_0 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}$$

NumPy computes the roots of $P(x)$ via `np.roots(c)` by computing the eigenvalues of this companion matrix using LAPACK's QR algorithm, delivering robust numerical convergence without initial guess divergence.

[illegible]

~~~
# 1. Prediction
x_pred = F @ x
P_pred = F @ P @ F.T + Q

# 2. Update (Kalman Gain)
S = H @ P_pred @ H.T + R
K = P_pred @ H.T @ np.linalg.inv(S)
y = z - H @ x_pred # Innovation residual
x_new = x_pred + K @ y
P_new = (np.eye(2) - K @ H) @ P_pred
return x_new, P_new
```
~~~

>> **Solution to Question 22:** [illegible]

Given N-point DFT $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{k n}$ where $W_N = e^{-j \frac{2\pi}{N}}$ and N is a power of 2.

Split $x[n]$ into even-indexed samples $x[2m]$ and odd-indexed samples $x[2m+1]$ ($m = 0, \dots, N/2 - 1$):
$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k(2m)} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$
Noting that $W_N^{2 m k} = W_{N/2}^{m k}$:
$$X[k] = E[k] + W_N^k O[k]$$
$$X[k + N/2] = E[k] - W_N^k O[k]$$
where $E[k]$ and $O[k]$ are $N/2$-point DFTs.

Computational Complexity Savings:
- Direct DFT: N^2 complex multiplications.
- Radix-2 FFT: $\frac{N}{2} \log_2 N$ complex multiplications.
- For $N = 1024$:
- Direct DFT: $1024^2 = 1,048,576$ multiplications.
- FFT: $512 \times 10 = 5,120$ multiplications.
- Computational Speedup: $> 200\times$ reduction in arithmetic operations!

[illegible]

[Detailed Step-by-Step Answer]:
Low-Rank Approximation via Eckart-Young-Mirsky Theorem:

Any noisy data matrix $A \in \mathbb{R}^{m \times n}$ can be decomposed via SVD: $A = \sum_{i=1}^r \sigma_i u_i v_i^T$.

```python
import numpy as np

def denoise_tsvd(A, k_components):
U, s, Vt = np.linalg.svd(A, full_matrices=False)
# Keep only the top k dominant singular values
s_filtered = np.zeros_like(s)
s_filtered[:k_components] = s[:k_components]

# Reconstruct optimal rank-k low-noise matrix
A_denoised = (U * s_filtered) @ Vt
return A_denoised
```

This filters out unstructured additive noise that populates the smaller tail singular values while preserving core correlated signal features.

>>Solution to Question 26:

[Problem Statement]: Implement a vectorized Digital Phase-Locked Loop (DPLL) phase error detector and loop filter in NumPy.

```
return phase_out
```

[illegible]

>> Solution to Question 30: Problem Statement: [illegible] Least Mean Squares (LMS) adaptive filter [illegible] hardware SIMD acceleration.

```
# Stochastic gradient descent weight update: w = w + 2*mu*e*x
weights += 2 * mu * e * x_window

return error, weights
```
